

Лабораторная работа №2

Блочный шифр BelT (СТБ 34.101.31-2011): режимы работы и имитовставка

Пояснительная записка к защите. Реализация: lab2/src, язык Rust

Содержание

1. Что сделано в работе	2
2. Аббревиатуры и обозначения	2
3. Паспорт алгоритма	3
4. Устройство шифра	4
4.1. Таблица H и функция G	4
4.2. Такт зашифрования	4
4.3. Такт расшифрования	5
4.4. Ключевое расписание	5
5. Режимы работы	5
5.1. Простая замена, <code>belt-ecb</code> (п. 6.2)	5
5.2. Сцепление блоков, <code>belt-cbc</code> (п. 6.3)	6
5.3. Счётчик, <code>belt-ctr</code> (п. 6.5)	6
5.4. Гаммирование с обратной связью, <code>belt-cfb</code> / CFM (п. 6.4)	6
5.5. Имитовставка, <code>belt-mac</code> (п. 6.6)	7
5.6. Сводная таблица режимов	7
6. Реализация	8
6.1. Структура проекта	8
6.2. Формат шифртекста	8
6.3. Проверки перед расшифрованием	8
6.4. Порядок байтов	9
7. Проверка работоспособности	9
8. Вопросы на защите	9

1. Что сделано в работе

Реализован белорусский стандарт блочного шифрования BelT и режимы его применения. Обязательная часть работы – режим простой замены и режим гаммирования с обратной связью; сцепление блоков и счётчик сделаны сверх задания и тоже заведены в меню.

Пункт стандарта	Режим	Методы BelT	Контрольные примеры
п. 6.1	Блочное преобразование F, F^{-1}	encrypt_block / decrypt_block	A.1, A.2
п. 6.2	belt-ecb – простая замена	encrypt_ecb / decrypt_ecb	A.6–A.9
п. 6.3	belt-cbc – сцепление блоков	encrypt_cbc / decrypt_cbc	A.10–A.13
п. 6.4	belt-cfb – гаммирование с обратной связью	encrypt_cfm / decrypt_cfm	A.14, A.15
п. 6.5	belt-ctr – счётчик	encrypt_ctr / decrypt_ctr	A.16
п. 6.6	belt-mac – имитовставка	compute_mac	A.17, A.18

Все режимы сверены с контрольными примерами приложения А стандарта – векторы лежат прямо в `#[cfg(test)] mod tests` внутри `lab2/src/encryption/bel_t.rs` и проверяются командой `cargo test -p lab2`. Хэширование (`belt-hash`, п. 6.9) в задание не входит и в программе не реализовано, поэтому в меню его нет.

Меню программы:

```
BelT (STB 34.101.31-2011) -- input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit):
```

2. Аббревиатуры и обозначения

Сокращение	Расшифровка	Значение
СТБ	Стандарт Беларуси	Государственный стандарт Республики Беларусь. Полное имя: СТБ 34.101.31-2011 «Информационные технологии и безопасность. Защита информации. Криптографические алгоритмы шифрования и контроля целостности». Введён 2011-07-01.
BelT	Belarusian Standard	Название алгоритма; в тексте стандарта режимы именуются <code>belt-ecb</code> , <code>belt-cbc</code> , <code>belt-ctr</code> , <code>belt-cfb</code> , <code>belt-mac</code> , <code>belt-hash</code> .
ECB	Electronic Codebook	«Электронная кодовая книга» – режим простой замены : каждый блок преобразуется независимо.

CBC	Cipher Block Chaining	Сцепление блоков: перед зашифрованием блок складывается по XOR с предыдущим блоком шифртекста.
CTR	Counter	Режим счётчика (гаммирование): шифруется значение счётчика, результат накладывается на текст по XOR.
CFB	Cipher Feedback	Гаммирование с обратной связью: гамма вырабатывается из предыдущего блока шифртекста. В коде и в меню обозначен как CFM.
MAC	Message Authentication Code	Код аутентификации сообщения; в стандарте – имитовставка . 64 бита, вычисляется на ключе.
CTS	Ciphertext Stealing	Кража шифртекста: приём, позволяющий обработать неполный последний блок без дополнения, сохранив длину сообщения. Применяется в belt-ecb и belt-cbc .
IV	Initialization Vector	Вектор инициализации, в стандарте – синхропосылка S , 128 бит.

Математические обозначения стандарта:

Запись	Смысл
$X = X_1 \parallel \dots \parallel X_n$	сообщение, разбитое на блоки по 128 бит; последний блок может быть короче
$K = (K_1, \dots, K_8)$	ключ: восемь 32-битных слов, 256 бит
$F(\cdot), F^{-1}(\cdot)$	зашифрование и расшифрование одного блока
$u \oplus v$	побитовое исключающее ИЛИ (XOR)
$u \boxplus v, u \boxminus v$	сложение и вычитание 32-битных слов по модулю 2^{32}
$G_r(u)$	подстановка байтов через таблицу H и циклический сдвиг слова влево на r
$\text{Left}_r(u)$	первые r байт блока u
$\text{Right}_r(u)$	последние r байт блока u
$\langle i \rangle_{32}$	число i 32-битным словом
$\langle 1 \rangle_{128}$	единица как 128-битное число, то есть прибавление 1 к счётчику

Ловушка: что такое Left_r

Left_r – это **первые r байт байтовой строки**, а в числовом представлении блока (`u128::from_le_bytes`) первый байт – самый младший. Поэтому «взять Left_r » в коде – это маска по **младшим** $8r$ битам: `n & (u128::MAX >> (128 - 8r))`, функция `Belt::l`. Подтверждается примерами из п. 5 стандарта: сдвиг «вниз» выбрасывает первый байт строки. Спутать направление – самая частая ошибка при реализации.

3. Паспорт алгоритма

Параметр	Значение
Год, статус	2011, действующий стандарт Республики Беларусь
Тип	симметричный блочный шифр
Структура	SP-сеть на четырёх 32-битных словах (схема типа Лэя–Мэсси), не сеть Фейстеля

Размер блока	128 бит (16 байт)
Размер ключа	128, 192 или 256 бит; в работе используется 256
Число тактов (раундов)	8, по 7 подключей на такт – всего 56 подключевых операций
Нелинейность	одна таблица H : подстановка 8 бит в 8 бит
Ключевое расписание	тривиальное: слова ключа берутся циклически, $K_{(7i-j) \bmod 8}$

Чем BelT отличается от ГОСТ 28147-89

- Блок 128 бит вместо 64 – снят порог «коллизии блоков после 2^{32} блоков».
- Не сеть Фейстеля: за один такт меняются все четыре слова, поэтому 8 тактов хватает там, где ГОСТу нужно 32 раунда.
- Таблица подстановки одна и на 8 бит (256 значений), а не восемь по 4 бита, и она **зафиксирована** в стандарте, а не оставлена на усмотрение ведомства.
- Расшифрование – отдельная процедура (такты идут в обратном порядке с другим распределением подключей), а не тот же код с перевёрнутым ключом.
- Неполный последний блок обрабатывается кражей шифртекста, а не дополнением, поэтому длина сообщения сохраняется даже в ECB и CBC.

4. Устройство шифра

4.1. Таблица H и функция G

Единственный нелинейный элемент – таблица H из 256 байт (п. 6.1.2). Функция G_r применяет её к каждому байту 32-битного слова и циклически сдвигает результат влево на r бит:

$$G_r(u) = \text{ROL}_r(H(u_{(0)}) \parallel H(u_{(1)}) \parallel H(u_{(2)}) \parallel H(u_{(3)}))$$

Используются три варианта: G_5 , G_{13} , G_{21} . Разные величины сдвига нужны, чтобы биты расходились по слову неодинаково и диффузия шла быстрее.

```
fn g(r: u32, u: u32) -> u32 {
    return u32::from_le_bytes(u.to_le_bytes().map(|byte| H_TABLE[byte as usize]))
        .rotate_left(r);
}
```

4.2. Такт зашифрования

Блок X представляется четырьмя 32-битными словами a, b, c, d (слово a – первые четыре байта). Такт с номером $i = 1, \dots, 8$ (п. 6.1.3):

1. $b \leftarrow b \oplus G_5(a \boxplus K_{7i-6})$
2. $c \leftarrow c \oplus G_{21}(d \boxplus K_{7i-5})$
3. $a \leftarrow a \boxplus G_{13}(b \boxplus K_{7i-4})$
4. $e \leftarrow G_{21}(b \boxplus c \boxplus K_{7i-3}) \oplus \langle i \rangle_{32}$
5. $b \leftarrow b \boxplus e$
6. $c \leftarrow c \boxplus e$
7. $d \leftarrow d \boxplus G_{13}(c \boxplus K_{7i-2})$
8. $b \leftarrow b \oplus G_{21}(a \boxplus K_{7i-1})$
9. $c \leftarrow c \oplus G_5(d \boxplus K_{7i})$
10. $a \leftrightarrow b, \quad c \leftrightarrow d, \quad b \leftrightarrow c$

После восьмого такта результат собирается в порядке

$$Y = b \parallel d \parallel a \parallel c$$

Номер такта $\langle i \rangle_{32}$ на шаге 4 – это **раундовая константа**: без неё все восемь тактов были бы одинаковыми и шифр стал бы уязвим к слайдовым атакам.

4.3. Такт расшифрования

Расшифрование (п. 6.1.4) идёт тактами $i = 8, 7, \dots, 1$, причём подключи внутри такта распределены зеркально:

1. $b \leftarrow b \oplus G_5(a \boxplus K_{7i})$
2. $c \leftarrow c \oplus G_{21}(d \boxplus K_{7i-1})$
3. $a \leftarrow a \boxplus G_{13}(b \boxplus K_{7i-2})$
4. $e \leftarrow G_{21}(b \boxplus c \boxplus K_{7i-3}) \oplus \langle i \rangle_{32}$
5. $b \leftarrow b \boxplus e, \quad c \leftarrow c \boxminus e$
6. $d \leftarrow d \boxplus G_{13}(c \boxplus K_{7i-4})$
7. $b \leftarrow b \oplus G_{21}(a \boxplus K_{7i-5})$
8. $c \leftarrow c \oplus G_5(d \boxplus K_{7i-6})$
9. $a \leftrightarrow b, \quad c \leftrightarrow d, \quad a \leftrightarrow d$

$$X = c \parallel a \parallel d \parallel b$$

Обратите внимание: шаги 4–6 (вычисление e и сложение/вычитание с b, c) при расшифровании **не** меняют знак. Это не опечатка: операция $(b, c) \rightarrow (b + e, c - e)$ вместе с последующей перестановкой слов обратна сама себе в нужном порядке – в этом и состоит схема Лэя–Мэсси. Именно эта симметрия сначала сбивает с толку при реализации.

4.4. Ключевое расписание

256-битный ключ режется на восемь слов $K_1, \dots, K_8$ (слово j – байты $4(j-1) \dots 4j$, little-endian). Никаких преобразований над ними нет: индекс берётся по модулю восьми,

$$K_m = \text{key}[(m-1) \bmod 8]$$

За 8 тактов по 7 подключей выходит 56 обращений, то есть каждое слово ключа используется семь раз. Для ключей 128 и 192 бит стандарт доопределяет расширение (п. 7.1, `belt-keyexpand`) – в работе оно не требуется, ключ всегда 256-битный.

```
fn get_subkey(key: &Key<8>, idx: u32) -> u32 {
    return key[(idx % 8) as usize];
}
```

5. Режимы работы

5.1. Простая замена, `belt-ecb` (п. 6.2)

Если длина сообщения кратна 16 байтам, режим тривиален:

$$Y_i = F(X_i), \quad X_i = F^{-1}(Y_i), \quad i = 1, \dots, n$$

Если последний блок неполный (r байт, $0 < r < 16$), применяется **кража шифртекста**: вместо того чтобы дополнять сообщение, у предпоследнего блока «занимают» недостающие байты.

$$\begin{aligned} Y_i &= F(X_i), \quad i = 1, \dots, n-2 \\ Y^* &= F(X_{n-1}) \\ Y_n &= \text{Left}_r(Y^*) \\ Y_{n-1} &= F(X_n \parallel \text{Right}_{16-r}(Y^*)) \end{aligned}$$

То есть: предпоследний блок шифруется, его первые r байт уезжают в конец шифртекста и становятся последним (коротким) блоком, а «хвост» склеивается с неполным блоком открытого текста и шифруется ещё раз. Расшифрование выполняется той же схемой с F^{-1} – в коде это одна функция `apply_ecb` с флагом направления.

Ограничение. Красть нечего, если сообщение короче одного блока, поэтому `belt-ecb` требует $|X| \geq 16$ байт. В меню это выражено проверкой `cli::at_least(BLOCK_SIZE)`, и пользователь получает понятное сообщение об ошибке, а не панику.

Свойства. Длина сохраняется: $|Y| = |X|$. Синхропосылки нет, шифрование детерминировано, одинаковые блоки дают одинаковый шифртекст – те же претензии, что и к ECB в ГОСТе. Полный параллелизм, произвольный доступ. Ошибка в одном бите портит весь свой блок (а если она попала в один из двух последних блоков – оба).

5.2. Сцепление блоков, `belt-cbc` (п. 6.3)

$$\begin{aligned} Y_0 &= S \\ Y_i &= F(X_i \oplus Y_{i-1}) \\ X_i &= F^{-1}(Y_i) \oplus Y_{i-1} \end{aligned}$$

Неполный последний блок снова обрабатывается кражей шифртекста:

$$\begin{aligned} Y^* &= F(X_{n-1} \oplus Y_{n-2}) \\ Y_n &= \text{Left}_r(Y^*) \\ Y_{n-1} &= F((X_n \parallel 0\dots 0) \oplus Y^*) \end{aligned}$$

Свойства. Зашифрование строго последовательно, расшифрование параллелится. Одинаковые блоки открытого текста дают разные блоки шифртекста. Ошибка в бите Y_i портит весь блок X_i и ровно тот же бит в X_{i+1} – дальше расшифрование восстанавливается.

Из-за кражи шифртекста минимальная длина – один блок, а шифртекст в программе несёт синхропосылку впереди, поэтому проверка при расшифровании – `at_least(2 * BLOCK_SIZE)`.

5.3. Счётчик, `belt-ctr` (п. 6.5)

$$\begin{aligned} s &= F(S) \\ s_i &= s \boxplus \langle i \rangle_{128} \\ Y_i &= X_i \oplus F(s_i), \quad Y_n = X_n \oplus \text{Left}_r(F(s_n)) \end{aligned}$$

Синхропосылка сначала **зашифровывается**, и счётчик наращивается уже от результата – иначе близкие синхропосылки давали бы пересекающиеся потоки гаммы.

Свойства. Дополнения нет, длина сохраняется ($|Y| = 16 + |X|$ с учётом открыто передаваемой синхропосылки). Обратное преобразование F^{-1} не используется. Гамма считается заранее и параллельно, поддерживается произвольный доступ к блокам. Ошибка в бите шифртекста портит ровно тот же бит открытого текста и не размножается.

5.4. Гаммирование с обратной связью, `belt-cfb` / `CFM` (п. 6.4)

Регистр обратной связи – предыдущий блок **шифртекста**:

$$\begin{aligned} Y_0 &= S \\ \text{зашифрование: } Y_i &= X_i \oplus \text{Left}_{|X_i|}(F(Y_{i-1})) \\ \text{расшифрование: } X_i &= Y_i \oplus \text{Left}_{|Y_i|}(F(Y_{i-1})) \end{aligned}$$

Единственная разница между направлениями – что уходит в регистр: при зашифровании результат XOR, при расшифровании входной блок. В обоих случаях вызывается только `encrypt_block`. Неполный блок всегда последний, регистр после него уже не нужен.

```
fn apply_cfm(bytes: &[u8], key: &Key<8>, sync: u128, encrypt: bool) -> Vec<u8> {
    let mut prev: u128 = sync;
    let mut result: Vec<u8> = Vec::with_capacity(bytes.len());

    for block in bytes.chunks_exact(BLOCK_SIZE) {
        let x = Self::block_from(block);
        let y = x ^ Self::encrypt_block(prev, key);
        result.extend_from_slice(&y.to_le_bytes());
    }
}
```

```

    prev = if encrypt { y } else { x };
  }
  // ... неполный последний блок: x_n ^ Left_r(F(prev))
  return result;
}

```

Свойства. Длина сохраняется, дополнения нет, F^{-1} не используется. Зашифрование строго последовательно, расшифрование распараллеливается. Искажение бита в Y_i портит тот же бит в X_i и весь блок X_{i+1} целиком, после чего поток восстанавливается – самосинхронизация через один блок. Это главное отличие от CTR и главный практический смысл режима.

5.5. Имитовставка, **belt-mac** (п. 6.6)

$$\begin{aligned}
 r &= F(0^{128}), \quad s = 0^{128} \\
 s &\leftarrow F(s \oplus X_i), \quad i = 1, \dots, n-1 \\
 s &\leftarrow \begin{cases} F(s \oplus X_n \oplus \varphi_1(r)) & \text{если } |X_n| = 128 \text{ бит} \\ F(s \oplus \psi(X_n) \oplus \varphi_2(r)) & \text{иначе} \end{cases} \\
 I &= \text{Left}_8(s) \quad (64 \text{ бита})
 \end{aligned}$$

Вспомогательные преобразования над блоком $u = u_1 \parallel u_2 \parallel u_3 \parallel u_4$ (по 32 бита):

$$\begin{aligned}
 \varphi_1(u) &= u_2 \parallel u_3 \parallel u_4 \parallel (u_1 \oplus u_2) \\
 \varphi_2(u) &= (u_1 \oplus u_4) \parallel u_1 \parallel u_2 \parallel u_3
 \end{aligned}$$

Это сдвиговый регистр с обратной связью и его обращение: $\varphi_2(\varphi_1(u)) = u$. Дополнение ψ – это добавление байта `0x80` и нулей до 16 байт (бит «1», за которым идут нули).

Зачем два разных φ_1 и φ_2

Если бы полный и неполный последние блоки обрабатывались одинаково, сообщение X и сообщение $X \parallel 0x80 \parallel 0\dots 0$ давали бы одну имитовставку – тривиальная подделка. Разные маскирующие значения $\varphi_1(r)$ и $\varphi_2(r)$ разделяют эти два случая. Приём известен как СМАС/ОМАС: два независимых подключа, полученных из $F(0)$, для дополненного и недополненного варианта.

Свойства. Имитовставка определена для сообщения **любой** длины, включая пустое (пустое сообщение – это один пустой неполный блок, который дополняется до `80 00 ... 00`). Полная длина – 64 бита; в меню её можно усесть, тогда берутся **первые** биты. Вероятность угадать усечённую до r бит имитовставку – 2^{-r} .

Как имитовставка представлена в коде

Байтовая строка $\text{Left}_8(s)$ упаковывается в `u64` так, чтобы **первый байт стал старшим разрядом**: тогда печать числа в hex совпадает с байтовой строкой стандарта, а усечение – это сдвиг вправо, то есть сохранение ведущих битов:

```

fn truncate_mac(s: u128, mac_bits: usize) -> u64 {
    let tag = u64::from_be_bytes(s.to_le_bytes()[..8].try_into().unwrap());

    return tag >> (64 - mac_bits);
}

```

Проверяется примерами A.17/A.18: полная имитовставка `2DAB59771B4B16D0`, усечённая до 32 бит – `7260DA60`, до 16 – `7260`.

5.6. Сводная таблица режимов

Свойство	ECB	CBC	CTR	CFM
Нужна синхропосылка	нет	да	да	да

Дополнение	нет (CTS)	нет (CTS)	нет	нет
Минимальная длина	16 байт	16 байт	любая	любая
Длина шифртекста	$ X $	$16 + X $	$16 + X $	$16 + X $
Используется F^{-1}	да	да	нет	нет
Одинаковые блоки X	дают одинаковые Y	дают разные Y	дают разные Y	дают разные Y
Зашифрование параллелизуется	да	нет	да	нет
Расшифрование параллелизуется	да	да	да	да
Ошибка в бите Y_i портит	весь X_i	весь X_i и бит в X_{i+1}	один бит X_i	бит в X_i и весь X_{i+1}
Самосинхронизация	–	да	нет	да

СВС и CFM: зеркальные по распространению ошибки

В СВС искажённый блок шифртекста полностью портит **свой** блок открытого текста и один бит в следующем; в CFM – наоборот, один бит в своём блоке и весь следующий блок. Причина: в СВС повреждённый блок проходит через F^{-1} , а в CFM он идёт напрямую в XOR и лишь потом попадает в шифр.

6. Реализация

6.1. Структура проекта

Файл	Содержимое
lab2/src/encryption/bel_t.rs	таблица H , G_r , такты, пять режимов, имитовставка, тесты с векторами приложения А
lab2/src/main.rs	сборка меню: список режимов, проверки длин, параметры имитовставки
common/src/cli.rs	текстовое меню, файловый ввод-вывод, hex, разбор ключа – общее с работой №1
lab2/docs/modes.md	рабочие заметки по стандарту и полный список контрольных примеров

6.2. Формат шифртекста

Стандарт считает синхропосылку внешним параметром протокола. В работе принято то же соглашение, что и в работе №1: она вырабатывается случайно при каждом зашифровании и передаётся открыто первым блоком.

$$\text{шифртекст} = \underbrace{S}_{16 \text{ байт}} \parallel \underbrace{\text{тело}}_{|X| \text{ байт}}$$

ECB – единственный режим без синхропосылки, его шифртекст ровно такой же длины, как открытый текст.

6.3. Проверки перед расшифрованием

Меню ничего не знает про режимы: каждый режим отдаёт ему две необязательные проверки, а CLI только зовёт их и подписывает ошибку именем режима.

Режим	Проверка	Почему
ECB	$ X \geq 16$ и $ Y \geq 16$	краже шифртекста нечего красть меньше чем из блока

CBC	$ X \geq 16, Y \geq 32$	то же плюс синхропосылка впереди
CTR, CFM	$ Y \geq 16$	в шифртексте обязана быть синхропосылка

6.4. Порядок байтов

Все слова – little-endian. Блок собирается через `u128::from_le_bytes`, слово ключа – через `u32::from_le_bytes`, поэтому hex-ключ из стандарта вводится в меню как есть, байт за байтом. Отсюда же и правило про Left, из раздела 2.

7. Проверка работоспособности

Тесты запускаются командой `cargo test -p lab2`, всего 25 тестов.

- Контрольные примеры стандарта.** Блочное преобразование (A.1, A.2), ECB на полных блоках и с кражей шифртекста в обе стороны (A.6–A.9), CBC (A.10–A.13), CFM (A.14, A.15), CTR (A.16), имитовставка (A.17, A.18) – совпадают побитово.
- Круговой прогон.** Для CBC, CTR и CFM на всех длинах сообщения от 0 до 80 байт: расшифрование восстанавливает открытый текст, длина шифртекста равна $16 + |X|$.
- Соккрытие повторов.** CBC на нулевом сообщении даёт разные блоки шифртекста; два зашифрования одного текста различаются, потому что синхропосылка новая.
- Распространение ошибки.** Инверсия одного бита шифртекста: CTR портит ровно один байт открытого текста; CFM – байт и весь следующий блок; CBC – блок и один байт в следующем. Это ровно то, что заявлено в сводной таблице.
- Имитовставка.** Усечение берёт ведущие биты; имитовставка есть у сообщения любой длины, включая пустое; дописывание нулевого байта меняет значение (то есть дополнение ψ действительно одно-значно).

Дополнительно прогонялась работа через само меню: файл шифровался в режиме ECB и в режиме CFM, затем расшифровывался, и результат побайтово совпал с исходным; размеры вышли $|X|$ и $16 + |X|$ соответственно.

8. Вопросы на защите

? Чем BelT принципиально отличается от ГОСТ 28147-89?

Блок 128 бит вместо 64; не сеть Фейстеля, а схема, меняющая все четыре слова за такт, поэтому тактов всего 8; одна таблица подстановки 8 в 8 бит вместо восьми таблиц 4 в 4, и она зафиксирована в стандарте; неполный блок обрабатывается кражей шифртекста, а не дополнением; расшифрование – отдельная процедура, а не тот же код с обратным порядком ключей.

? Почему в BelT хватает восьми тактов, а ГОСТу нужно 32 раунда?

В сети Фейстеля за раунд изменяется только половина блока, поэтому нужны десятки раундов. В такте BelT каждое из четырёх слов участвует в нескольких операциях и все они изменяются, так что полная диффузия достигается за два-три такта.

? Что такое кража шифртекста и зачем она нужна?

Приём, позволяющий зашифровать сообщение, не кратное блоку, **без дополнения**: недостающие до полного блока байты «занимаются» у зашифрованного предпоследнего блока. Выигрыш – длина шифртекста равна длине открытого текста, не нужно передавать служебные байты и не возникает неоднозначности дополнения. Плата – сообщение должно быть не короче одного блока, и два последних блока обрабатываются особым образом.

? Почему `belt-ecb` не принимает сообщения короче 16 байт?

Кража шифртекста берёт байты у предыдущего блока; если блок всего один и он неполный, брать неоткуда. Стандарт в этом случае режим не определяет. В программе стоит явная проверка, которая выдаёт ошибку вместо аварии.

? Почему в ECB и CBC используется F^{-1} , а в CTR и CFM нет?

В ECB и CBC через шифр проходит сам открытый текст, поэтому для восстановления нужна обратная

функция. В CTR и CFM шифр вырабатывает гамму, одну и ту же в обе стороны, а текст только складывается с ней по XOR – обратное преобразование не требуется. В аппаратуре это экономит половину схемы.

? Почему в CTR синхропосылка сначала шифруется, а уже потом наращивается?

Чтобы близкие или предсказуемые синхропосылки давали непересекающиеся потоки гаммы. Если наращивать сам S , то два сообщения с синхропосылками, отличными на единицу, получили бы почти одинаковую гамму со сдвигом на блок.

? Что будет, если повторить синхропосылку в CTR?

Гамма повторится, и XOR двух шифртекстов даст XOR двух открытых текстов – защита исчезает полностью, ключ для этого не нужен. Поэтому S берётся из генератора случайных чисел при каждом зашифровании: 128 случайных бит делают повтор невозможным на практике.

? В чём разница между CTR и CFM, если оба – гаммирование?

Источник гаммы. В CTR это счётчик, независимый от данных: гамму можно посчитать заранее, параллельно, и ошибка в шифртексте не размножается. В CFM гамма вырабатывается из предыдущего блока шифртекста: зашифрование последовательно, ошибка портит и следующий блок, зато режим самосинхронизируется после потери или вставки блока.

? Что значит самосинхронизация CFM?

Состояние режима – это последний переданный блок шифртекста. Если блок потерян, искажён или вставлен лишний, то через один блок регистр обратной связи снова содержит корректные данные, и расшифрование продолжается верно; безвозвратно теряется только один блок.

? Сравните распространение ошибки в CBC и CFM.

Они зеркальны. CBC: искажённый блок Y_i проходит через F^{-1} , поэтому X_i разрушен целиком, а в X_{i+1} испорчен ровно тот же бит. CFM: Y_i сначала складывается по XOR (портится один бит X_i), а потом идёт в шифр как регистр обратной связи – и разрушает весь X_{i+1} .

? Зачем в имитовставке значение $r = F(0)$ и два преобразования φ_1, φ_2 ?

r – это маскирующее значение, выведенное из ключа (шифрование нулевого блока). φ_1 и φ_2 дают из него две разные маски: одну для случая, когда последний блок полный, другую – когда он дополнялся. Без такого разделения сообщение и его дополненная версия давали бы одинаковую имитовставку, то есть подделка была бы тривиальной. Это стандартная конструкция CMAC/OMAC.

? Почему φ_1 и φ_2 взаимно обратны и важно ли это?

Обе – сдвиговый регистр с линейной обратной связью, взятый в двух направлениях: $\varphi_2(\varphi_1(u)) = u$. Для безопасности важно лишь то, что они дают **разные** и непредсказуемые без ключа маски; взаимная обратность – свойство выбранной конструкции, позволяющее задать обе одним регистром.

? Почему имитовставка определена даже для пустого сообщения?

Потому что пустое сообщение – это один неполный (нулевой длины) последний блок, который дополняется до `80 00 ... 00` и обрабатывается второй ветвью. Это отличие от реализации ГОСТа в работе №1, где пустое сообщение имитовставки не имеет.

? Что даёт усечение имитовставки и до сколько бит её можно резать?

Усечение экономит место в сообщении ценой стойкости: вероятность угадать усечённую до r бит имитовставку – 2^{-r} . Полная длина в BelT – 64 бита; в программе можно задать от 1 до 64, по умолчанию 64, и берутся **ведущие** биты.

? Зачем вообще имитовставка, если данные зашифрованы?

Шифрование даёт конфиденциальность, но не целостность. В CTR и CFM противник может инвертировать выбранные биты шифртекста и предсказуемо изменить открытый текст, не зная ключа. Имитовставка это обнаруживает, потому что вычисляется на секретном ключе. В самом стандарте для совместного решения обеих задач есть отдельный режим `belt-datawrap` (п. 6.7).

? Почему `Leftr` – это младшие биты числа, а не старшие?

Стандарт работает с байтовыми строками и нумерует байты с начала строки, а блок в программе представлен числом `u128`, собранным как little-endian, где первый байт строки – младший байт числа. Поэтому «первые r байт» – это маска по младшим $8r$ битам. Это чисто вопрос представления, а не математики.

? Как проверялось, что реализация правильная?

Контрольными примерами приложения А самого стандарта: для каждого режима в коде лежит тест с ключом, синхропосылкой, открытым текстом и ожидаемым шифртекстом из стандарта, включая случаи неполного последнего блока в обе стороны. Плюс круговые прогоны на всех длинах и проверки заявленных свойств режимов (распространение ошибки, зависимость от синхропосылки).

? Что ещё есть в стандарте, чего нет в работе?

`belt-datawrap` (п. 6.7) – совместное шифрование и имитозащита; `belt-keywrap` (п. 6.8) – то же для ключей; `belt-hash` (п. 6.9) – хэширование в 256 бит; `belt-keyexpand` (п. 7.1) – расширение 128- и 192-битных ключей до 256; `belt-keyrep` (п. 7.2) – преобразование ключа. Задание охватывает режимы шифрования и имитовставку.

? Какой режим вы бы выбрали для реальной задачи и почему?

Ни один из перечисленных сам по себе: они дают только конфиденциальность. Для реальной передачи нужен режим с аутентификацией – в терминах стандарта `belt-datawrap`, то есть шифрование (CTR) плюс имитовставка над шифртекстом. Из «чистых» режимов CTR удобнее всего (параллелится, не размножает ошибку, не требует дополнения), но обязателен уникальный S и обязательна имитовставка поверх.